SQL Transactions – Complete Notes

---

1. Introduction to SQL Transactions

An **SQL Transaction** groups one or more SQL operations (INSERT, UPDATE, DELETE) into a single unit of work.

It ensures:

- All operations succeed together
  OR
- None of them are applied

This preserves **data integrity** and reliability.

Why Transactions Are Important

- Ensures consistency by committing changes together
- Rolls back changes if any operation fails
- Prevents partial updates

Example: In a banking system, money transfer must either complete fully or not happen at all.

---

2. ACID Properties of SQL Transactions

SQL Transactions follow the **ACID Properties**:

1. Atomicity

- A transaction is fully completed or fully rolled back.
- If one part fails → entire transaction fails.

2. Consistency

- Database moves from one valid state to another.
- All rules and constraints are maintained.

3. Isolation

- Transactions execute independently.
- Simultaneous transactions do not interfere improperly.

4. Durability

- Once committed, changes are permanent.
- Data remains safe even after system failure.

---

3. SQL Transaction Control Commands

These commands control when changes are saved or undone.

1️⃣ BEGIN TRANSACTION

Starts a new transaction.

All commands after this are part of the same transaction until COMMIT or ROLLBACK.

Syntax

```
BEGIN TRANSACTION transaction_name;
```

Example: Bank Transfer Scenario

```
BEGIN TRANSACTION;

-- Deduct $150 from Account A
UPDATE Accounts
SET Balance = Balance - 150
WHERE AccountID = 'A';

-- Add $150 to Account B
UPDATE Accounts
SET Balance = Balance + 150
WHERE AccountID = 'B';

-- Commit if both succeed
COMMIT;
```

Explanation:

- Both UPDATE statements run together.
- If successful → COMMIT saves changes.
- If any fails → use ROLLBACK.

```
ROLLBACK;
```

2️⃣ COMMIT Command

Saves all changes permanently.

Once committed:

- Changes cannot be undone.

Syntax

COMMIT;

Example

```
DELETE FROM Student
WHERE AGE = 20;

COMMIT;
```

Result:

- Rows with AGE = 20 are permanently deleted.
- Changes are saved in the database.

---

3 ROLLBACK Command

Cancels all changes in the current transaction.

Restores database to the state before BEGIN TRANSACTION.

Syntax

ROLLBACK;

Example

```
DELETE FROM Student
WHERE AGE = 20;

ROLLBACK;
```

Result:

- Deletion is undone.
- Table returns to original state.

---

4 SAVEPOINT Command

A SAVEPOINT creates a checkpoint inside a transaction.

It allows partial rollback.

Syntax

SAVEPOINT savepoint_name;

Example

```
SAVEPOINT SP1;

DELETE FROM Student
WHERE AGE = 20;

SAVEPOINT SP2;
```

Explanation:

- SP1 created before delete.
- SP2 created after delete.
- You can rollback to SP1 or SP2.

---

5 ROLLBACK TO SAVEPOINT

Rolls back to a specific savepoint.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Example

```
ROLLBACK TO SP1;
```

Result:

- Delete operation is undone.
- Table returns to state at SP1.

---

6 RELEASE SAVEPOINT

Removes a savepoint.

After release:

- Cannot rollback to that savepoint.

Syntax

RELEASE SAVEPOINT savepoint_name;

Example

RELEASE SAVEPOINT SP2;

Result:

- SP2 is removed.
- No change to table data.
- Cannot rollback to SP2 anymore.

---

4. Types of SQL Transactions

1. Read Transactions

- Only read data.
- Use SELECT statements.

2. Write Transactions

- Modify data.
- Use INSERT, UPDATE, DELETE.

3. Distributed Transactions

- Span multiple databases.
- Maintain consistency across systems.

4. Implicit Transactions

- Automatically started by SQL Server.

5. Explicit Transactions

- Manually controlled.
- Use BEGIN, COMMIT, ROLLBACK.

---

5. Transaction Optimization Best Practices

To improve performance:

- Keep transactions small
- Avoid long-running transactions

- Monitor locks to prevent deadlocks
- Use batching for large data operations
- Commit as soon as possible

---

6. Real-Life Example (Banking System)

When transferring money:

1. Deduct from Account A
2. Add to Account B
3. Commit

If any step fails:

- Rollback entire transaction
- No money is lost
- Data remains consistent

---

7. Summary

Command	Purpose
BEGIN TRANSACTION	Starts transaction
COMMIT	Saves changes permanently
ROLLBACK	Cancels entire transaction
SAVEPOINT	Creates checkpoint
ROLLBACK TO SAVEPOINT	Partial undo
RELEASE SAVEPOINT	Removes savepoint

Final Quick Revision

- Transactions ensure safe data processing.
- Follow ACID properties.
- COMMIT saves.
- ROLLBACK cancels.
- SAVEPOINT allows partial undo.